

BAB IX

MEMBUAT CLIENT-SERVER

A. Capaian Pembelajaran

Pada pertemuan ini akan dijelaskan mengenai tutorial cara membuat aplikasi sederhana Client – Server dengan Java.

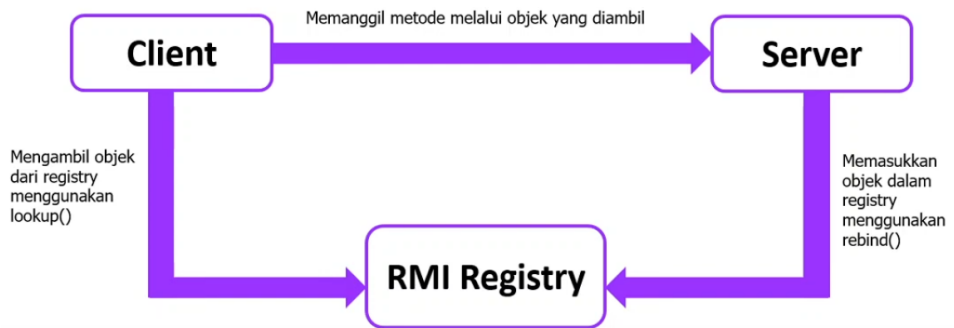
B. Materi

Istilah client/server pertama kali digunakan pada tahun 1980-an dalam mereferensikan suatu Personal Computer pada suatu jaringan. Model client/server yang sebenarnya mulai diterima pada akhir tahun 1980-an. Arsitektur perangkat lunak client/server adalah infrastruktur modular dan message-based yang sangat berguna, dimaksudkan untuk meningkatkan usability, flexibility, interoperability, dan scalability.

1. Pengertian RMI

RMI atau Remote Method Invocation adalah suatu cara agar client bisa melakukan invokasi atau

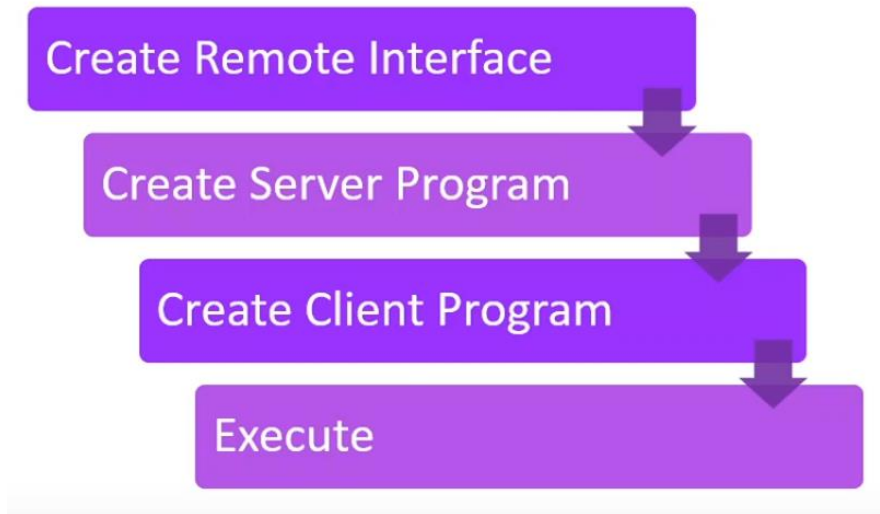
memanggil metode melalui objek yang ada di server, agar server bisa memberikan objek dan bisa dipanggil melalui metode oleh client maka diperlukan yang namanya RMI registry, sehingga server itu harus memasukkan objek dalam registry menggunakan metode `rebind()` sehingga nanti metode yang sudah dikemas dalam sebuah objek tadi itu bisa di ambil oleh sisi client dengan cara menggunakan method `lookup()`.



Gambar 93. Implementasi RMI

Langkah-langkah implementasi Java RMI :

- Membuat sebuah Remote Interface yang akan memberikan antarmuka bagi client dan server untuk saling bisa `rebind()` objek nya itu di registry dan client bisa memanggil objek yang ada di registry dengan `lookup()`.
- Membuat program Server
- Membuat program Client
- Execute



Gambar 94. langkah-langkah implementasi Java RMI

2. Membuat Program Server RMI

- Menyimpan Object ke Server

Untuk menyimpan Object ke Server kita bisa menggunakan perintah :

```
Data data = new Data();  
registry.bind("data", data);
```

Metode `bind()` tersebut mempunyai dua parameter, yang pertama `String` dan yang kedua `Object`. `String` yang dimaksud adalah nama `Object` yang disimpan, sesuai yang kita inginkan.

- Menghapus Object dari Server

Cara menghapus object dari server. Caranya dengan menggunakan metode `unbind(String nama)` milik `java.rmi.registry.Registry` :

```
registry.unbind("data");
```

- Membuat Client RMI

Setelah tadi kita membuat Server, sekarang saatnya kita membuat Client untuk mengakses object yang ada di Server. Berbeda dengan membuat Server, untuk membuat Client, kita menggunakan class `java.rmi.Naming` :

```
java.rmi.Naming
```

- Mengakses Object yang ada di Server

Untuk mengakses data yang ada pada server, kita harus membuat object Interface-nya, bukan classnya.

```
DataInterface data =
```

```
(DataInterface)
```

```
Naming.lookup("rmi://localhost:1099/data");
```

3. Menjalankan RMI

Setelah kita membuat Object yang akan diakses melalui RMI, lalu membuat Server dan membuat Client, sekarang kita akan coba menjalankan RMI.

- Menjalankan Server

Untuk menjalankan Server, kita hanya harus menggabungkan kode-kode yang telah menjadi bagian Membuat Server RMI kedalam class yang memiliki metode utama. Jika dijalankan maka akan terlihat trace dengan tulisan “Server Berhasil Berjalan”. Namun jika gagal biasanya kegagalan terjadi pada port.

- Menjalankan Client

Untuk menjalankan Client, kita harus menggabungkan seluruh kode pada bagian membuat Client RMI dalam satu class yang memiliki metode utama. Perlu diingat, sebelum menjalankan Client, pastikan bahwa Server telah berjalan. Jika Server telah berjalan coba jalankan Client (kode diatas). Maka dalam trace output program akan terlihat seperti ini :

Client Berhasil Terkoneksi Ke Server

Client Telah Selesai

Setelah itu lihat trace output pada Server, maka akan terlihat seperti ini :

Server Berhasil Berjalan

Metode Satu Dijalankan

Metode Dua Dijalankan

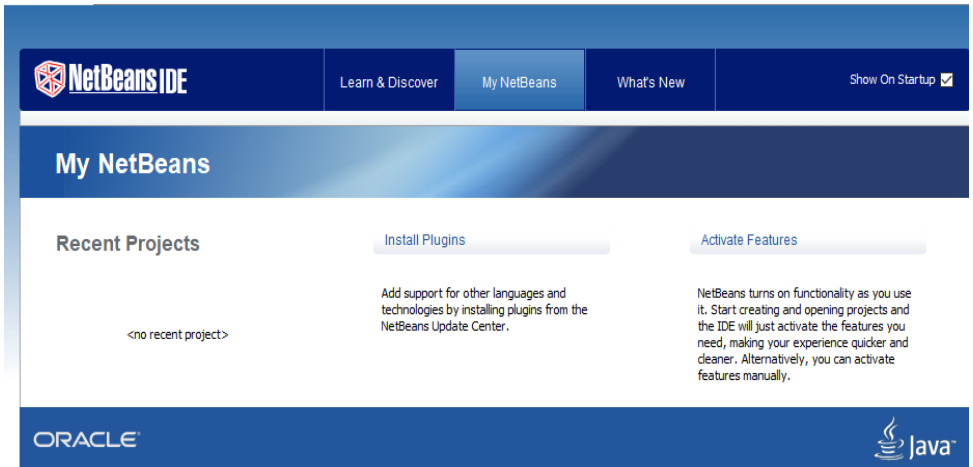
Dengan demikian kita telah berhasil membuat aplikasi sederhana berbasis Client Server, dimana Client memerintah sesuatu dan Server menjalankannya.

4. Cara membuat project Client-Server di NetBeans IDE

Dalam membuat project client server, kita akan membuat tiga project. Project pertama berisikan interface SayHello. Pada project ini berisikan seluruh class yang bersifat Interface atau dengan kata lain diperlukan oleh semua project Client maupun Server. Project kedua merupakan project Server, pada project ini berisikan seluruh class Server. Project ketiga merupakan project Client, dalam project ini berisikan seluruh class Client. Dan dalam project inilah kita bisa mengakses Server lewat RMI.

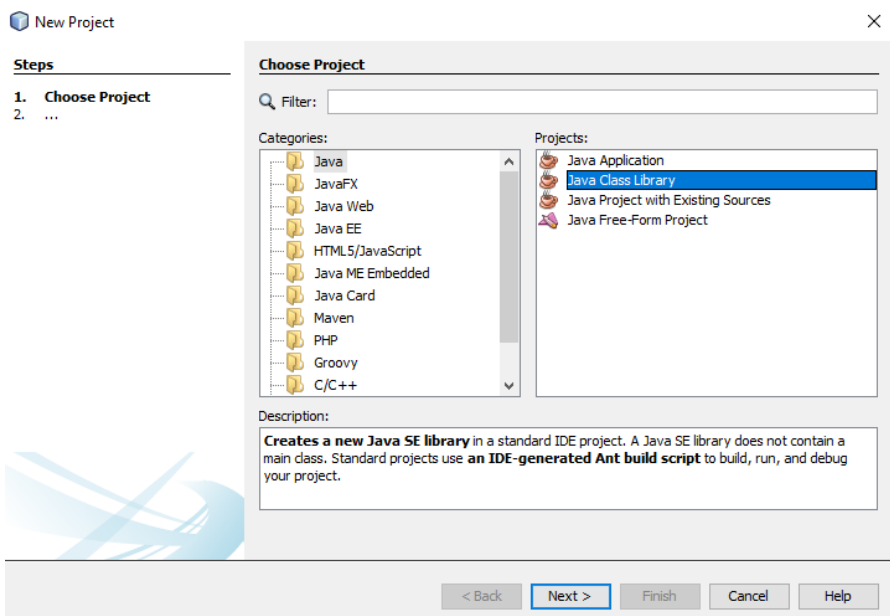
Langkah-langkah :

Pertama buka NetBeans IDE



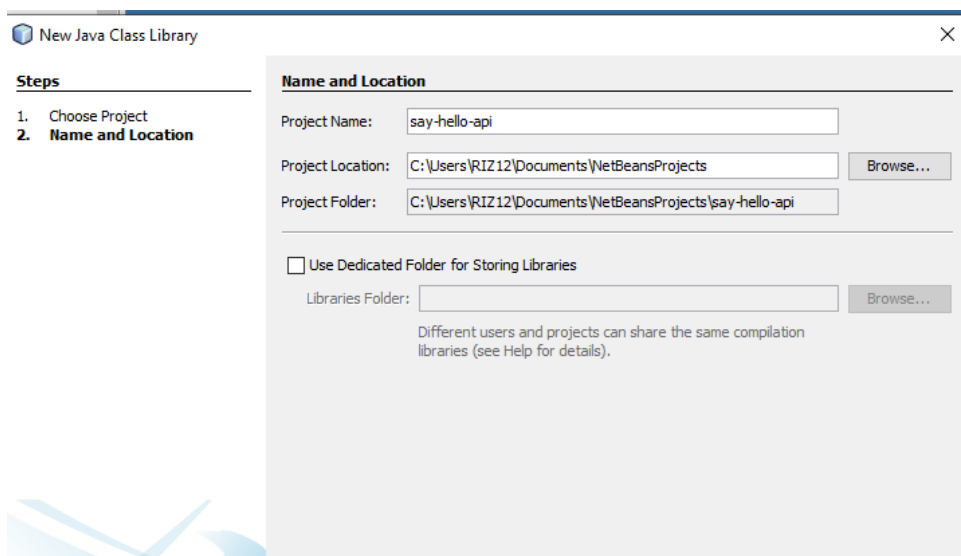
Gambar 95. Netbeans IDE

Lalu untuk membuat project pertama kita dapat memberi nama say-hello-api, pada project ini kita tidak membuat Aplikasi, jadi kita pilih Java > Java Class Library.



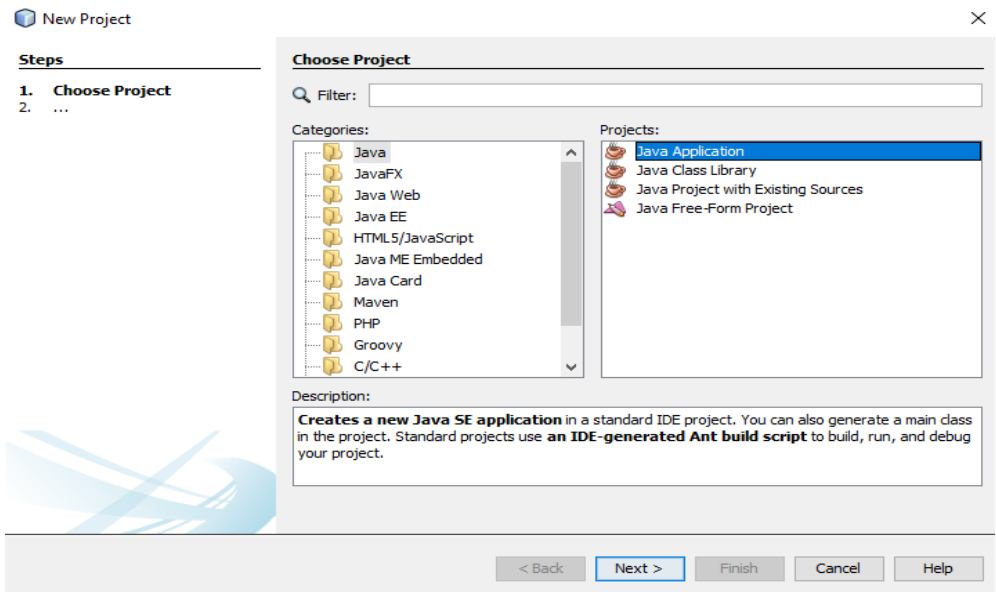
Gambar 96. Java Class Library

Lalu klik Next.



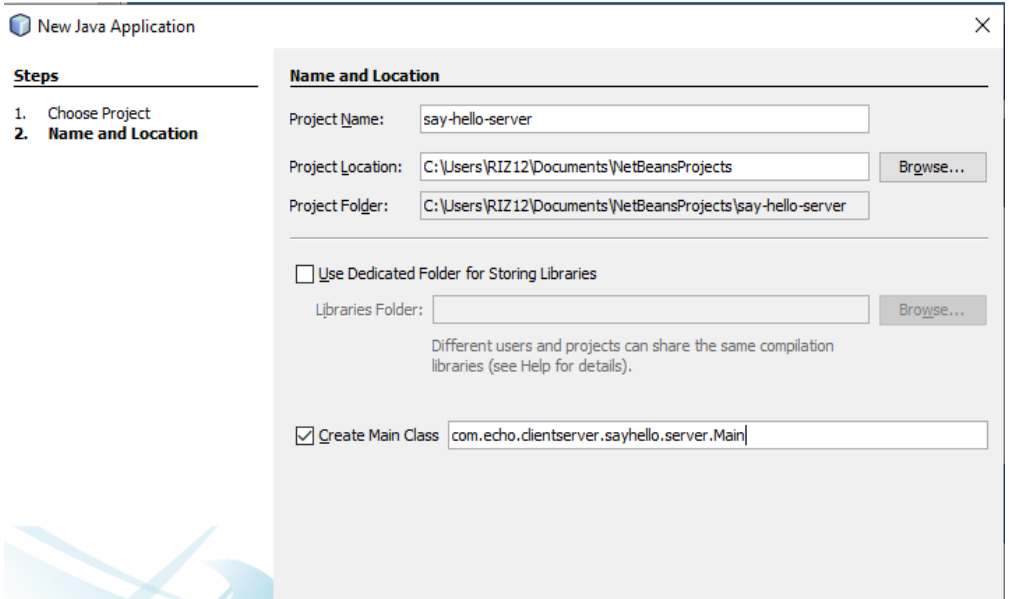
Gambar 97. New Java Class Library

Beri nama “say-hello-api” pada project name, lalu klik Finish. Setelah itu buat project yang kedua, New Project > Java > Java Application.



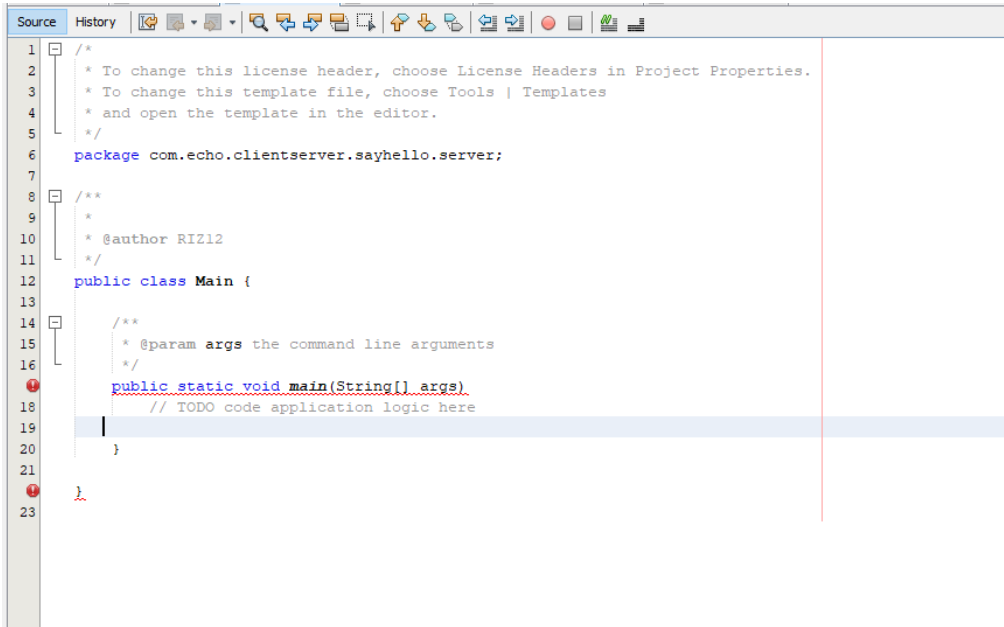
Gambar 98. Java Application

Lalu klik Next.



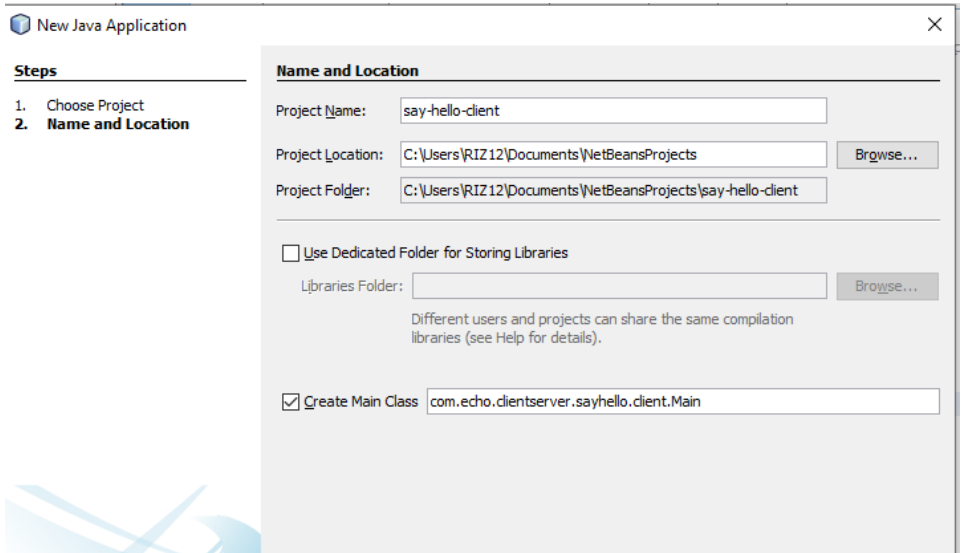
Gambar 99. New Java Application

Beri nama “say-hello-server” lalu pada Create Main Class beri tanda ceklist dan beri nama com.echo.clientserver.sayhello.Main. Klik Finish. Maka akan keluar gambar seperti dibawah.



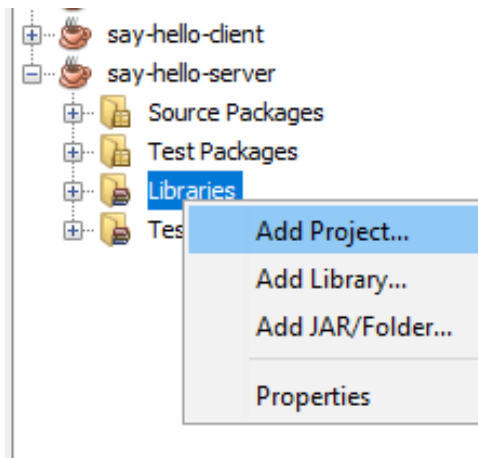
Gambar 100. Tampilan Default SayHello Server

Setelah itu kita buat project yang ketiga yaitu Client, caranya sama dengan membuat server, pilih New Project > Java > Java Application, klik Next.



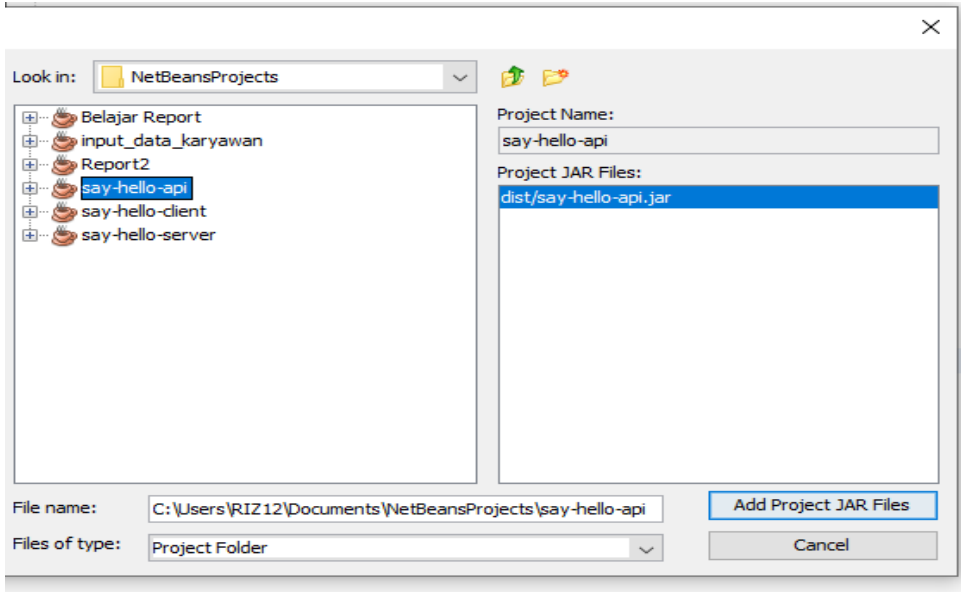
Gambar 101. New Java Appllication

Pada Project Name beri nama “say-hello-client” lalu pada Create Main Class “com.echo.clientserver.sayhello.client.Main”. Klik Finish.



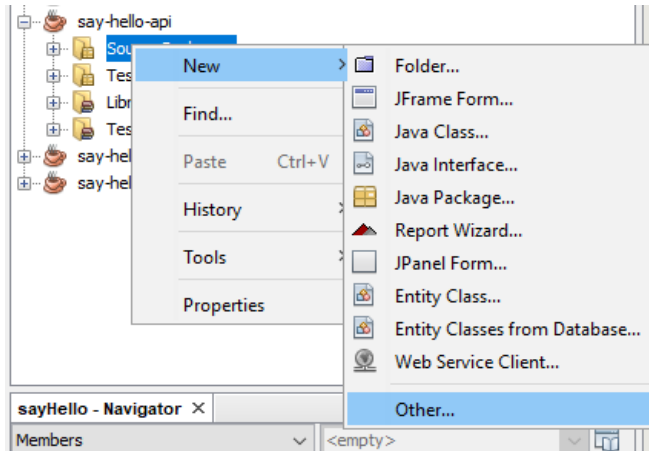
Gambar 102. Menambah Project pada Libraries

Pada bagian Libraries “say-hello-server” klik kanan lalu pilih Add Project. Maka akan keluar gambar seperti dibawah.



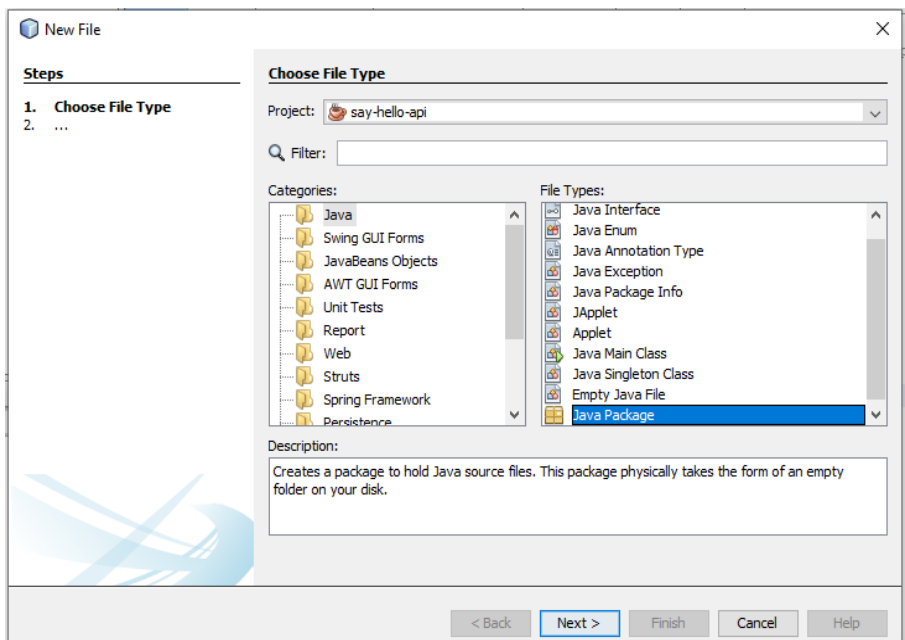
Gambar 103. Pilih project say-hello-api

Pilih project say-hello-api lalu klik Add Project JAR Files, lakukan hal yang sama pada project say-hello-client, masukkan project say-hello-api, caranya sama seperti diatas.



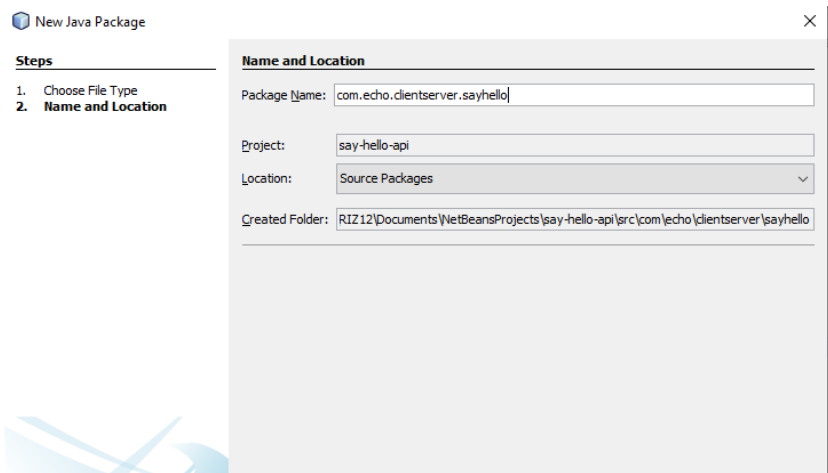
Gambar 104. New Package

Pada project say-hello-api buatlah package baru.
Klik kanan pada source packages pilih New > Other.



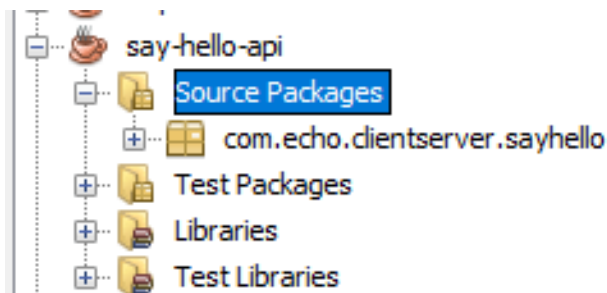
Gambar 105. New File

Lalu pilih Java > Java Package > Next.



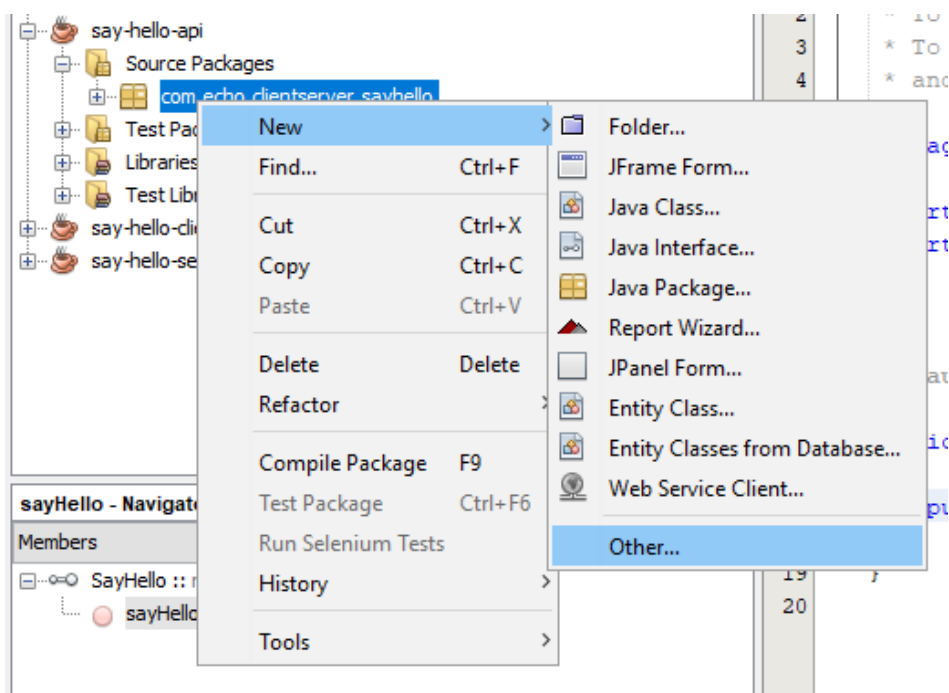
Gambar 106. New Java Package

Pada Project Name beri nama “com.echo.clientserver.sayhello, lalu klik Finish. Maka kita bisa lihat ada project package baru pada project say-hello-api.



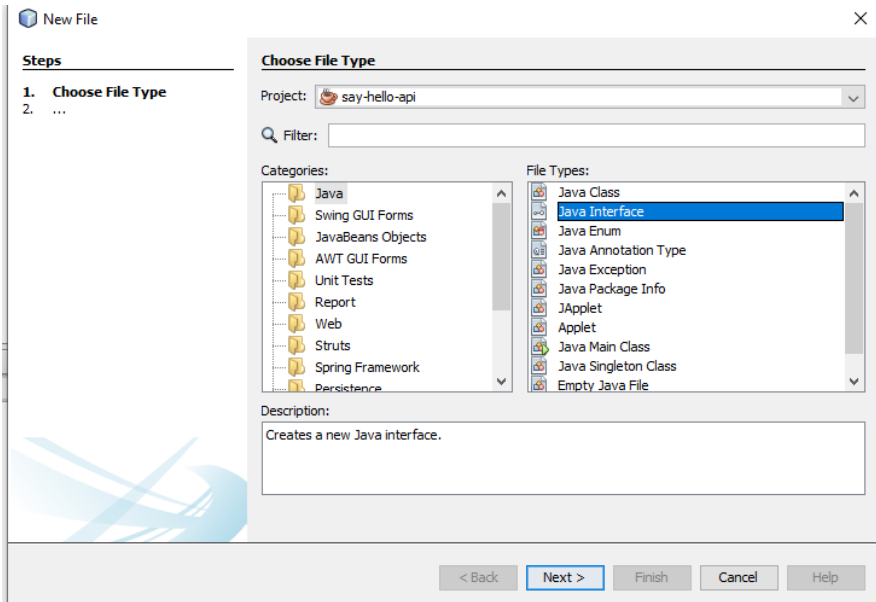
Gambar 107. Package Baru

Lalu kita buat Interface SayHello, klik kanan pada package yang kita buat, lalu pilih New > Other.



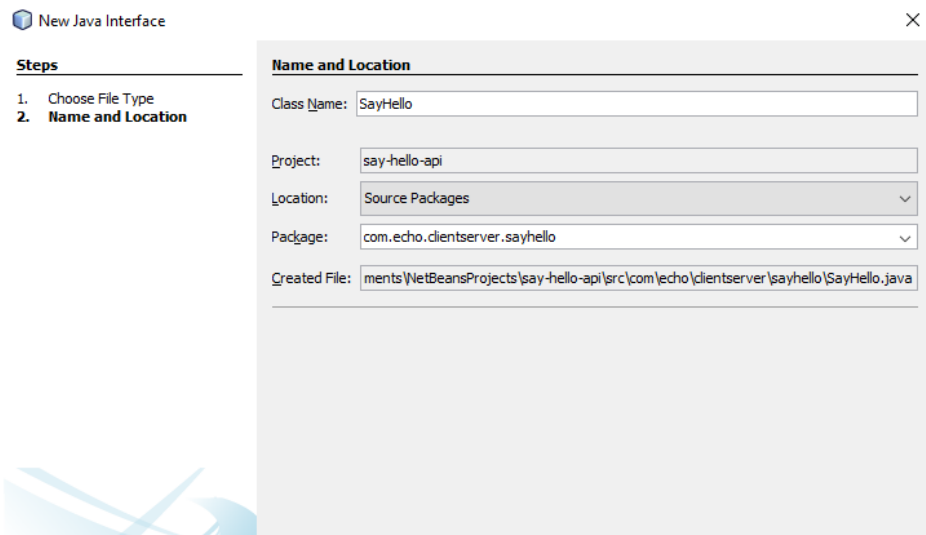
Gambar 108. New Interface

Pada Categories pilih Java dan File Types pilih Java Interface.



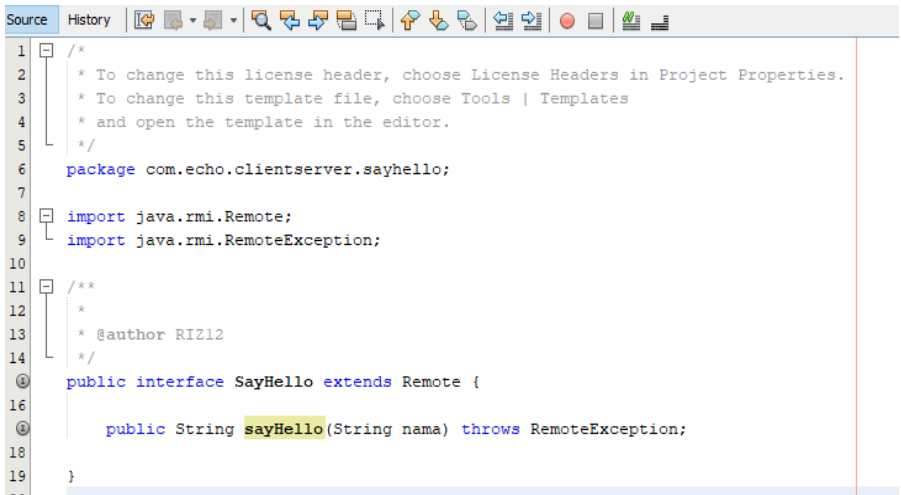
Gambar 109. New File

Lalu klik Next.



Gambar 110. New Java Package

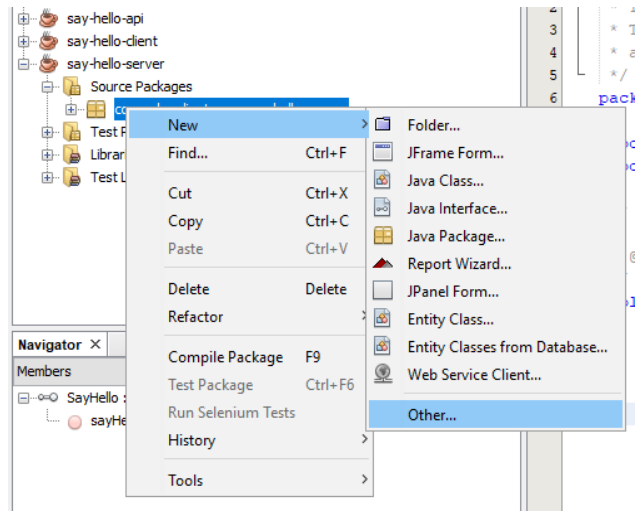
Pada Class Name beri nama SayHello untuk Interface-nya, pastikan pada Package harus “com.echo.clientserver.sayhello, lalu klik Finish. Dan akan muncul gambar seperti dibawah ini.

A screenshot of an IDE's source editor window. The window has a title bar with 'Source' and 'History' tabs. Below the tabs is a toolbar with various icons for editing and development. The main area shows a Java source file with the following code:

```
1  /*
2  * To change this license header, choose License Headers in Project Properties.
3  * To change this template file, choose Tools | Templates
4  * and open the template in the editor.
5  */
6  package com.echo.clientserver.sayhello;
7
8  import java.rmi.Remote;
9  import java.rmi.RemoteException;
10
11  /**
12   *
13   * @author RIZ12
14   */
15  public interface SayHello extends Remote {
16
17      public String sayHello(String nama) throws RemoteException;
18
19  }
```

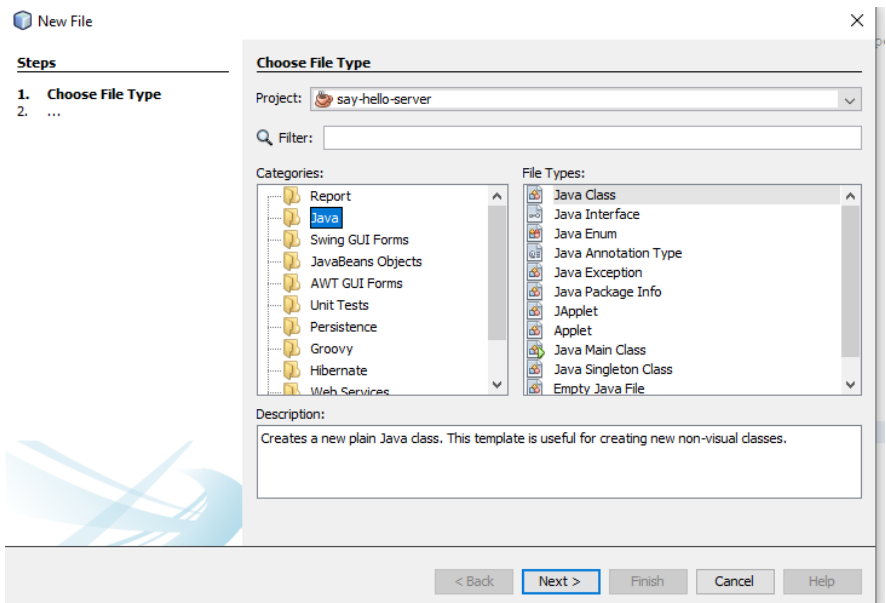
Gambar 111. Interface SayHello

Sekarang kita akan membuat class SayHelloServer, klik kanan pada package com.echo.clientserver.sayhello.server, lalu pilih New > Other.



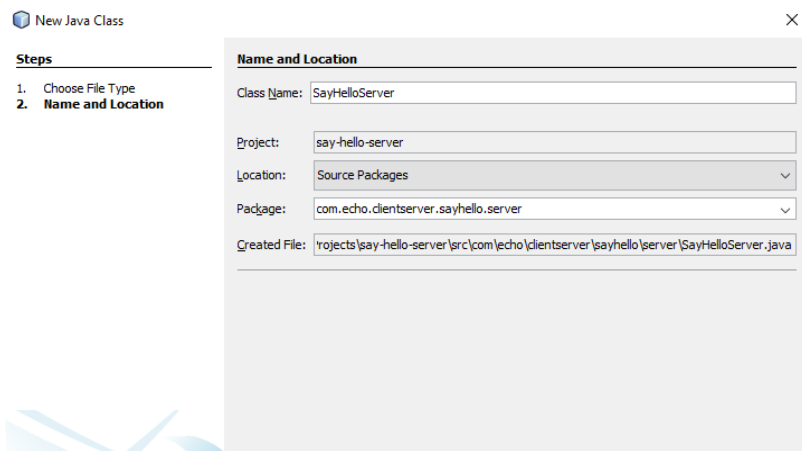
Gambar 112. New Class

Pilih Java > Java Class, Klik Next.



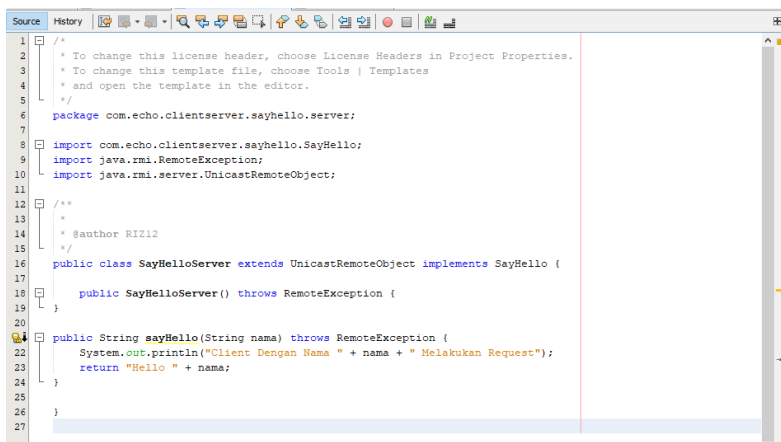
Gambar 113. New File

Setelah klik Next maka akan terlihat dialog seperti dibawah ini :



Gambar 114. New Java Class

Pada Class Name beri nama “SayHelloServer” lalu pastikan package nya com.echo.clientserver.sayhello.server, lalu klik Finish.



Gambar 115. Class SayHelloServer

Setelah itu isi class SayHelloServer dengan kode seperti yang telah kita buat.

a. Membuat Server SayHello

Untuk membuat Server SayHello, kita akan membuatnya di class Main yang otomatis dibuat oleh NetBeans. Dan Server ini kita buat pada project say-hello-server. Untuk membuat server berarti kita membuat sebuah Registry, setelah itu simpan object yang akan di akses melalui RMI di Registry :

```
package com.echo.clientserver.sayhello.server;

import java.rmi.RemoteException;

import java.rmi.registry.LocateRegistry;

import java.rmi.registry.Registry;

public class Wain {

    public static void main(String[] args) throws
    RemoteException {

        Registry registry = LocateRegistry.createRegistry(1099);

        SayHelloServer sayHello = new SayHelloServer();

        registry.rebind("sayHello", sayHello);

        System.out.println("Server Telah Berjalan");

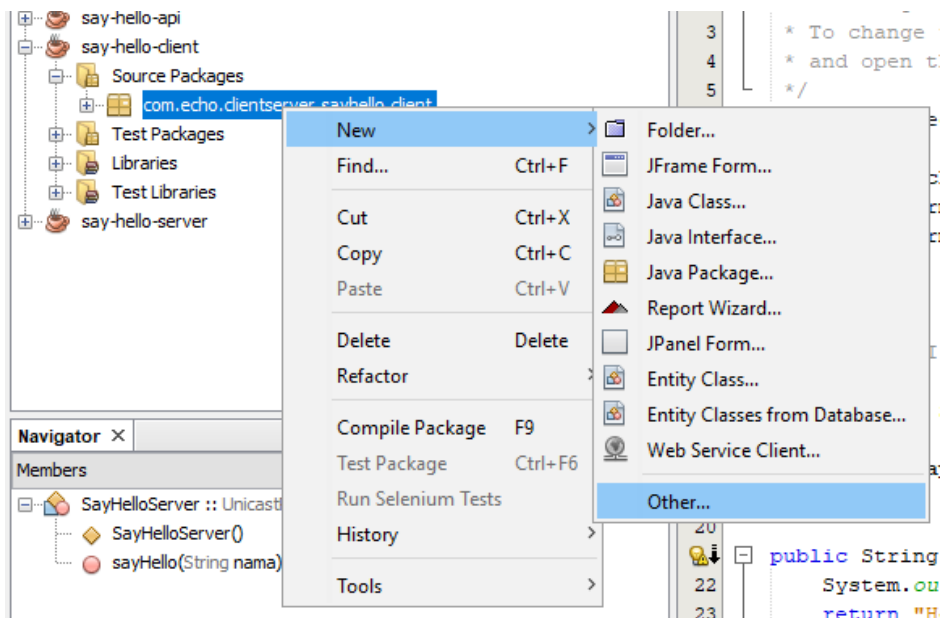
    }

}
```

Untuk membuat Server memang tidak terlalu rumit, karena tidak perlu membuat tampilan GUI nya, berbeda dengan Client, biasanya di sisi Client kita harus membuat tampilan GUI agar mempermudah User dalam menjalankan Aplikasinya.

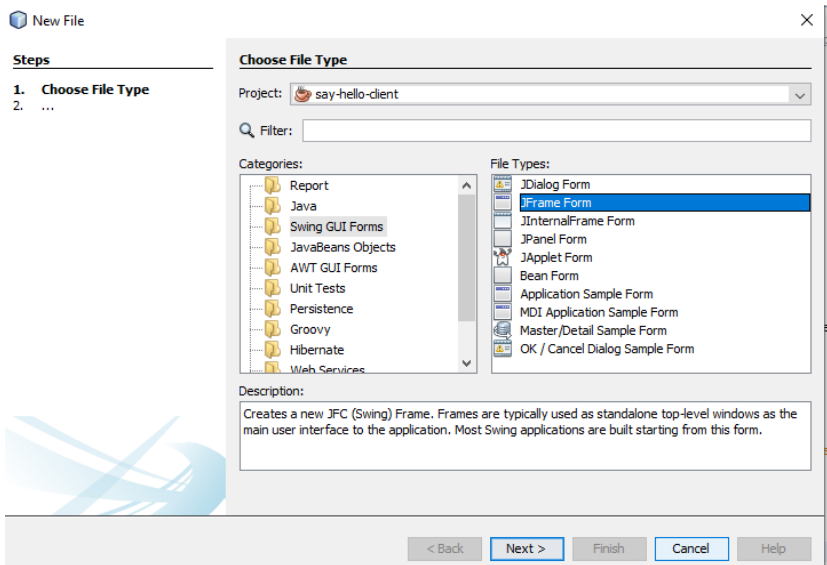
b. Membuat Client SayHello

Karena pada sisi Client kita ingin dalam bentuk tampilan GUI, maka kita harus membuat Form. Untuk membuat JFrame dengan NetBeans caranya klik kanan pada package `com.echo.clientserver.sayhello.client` pada project `say-hello-client`, pilih `New > Other`,



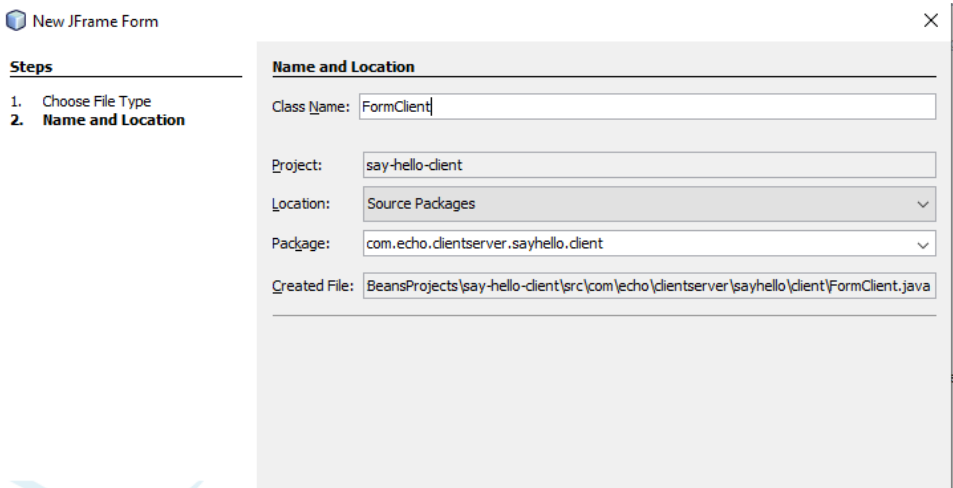
Gambar 116. New JFrame

Pilih Swing Gui Forms, lalu JFrame Form.



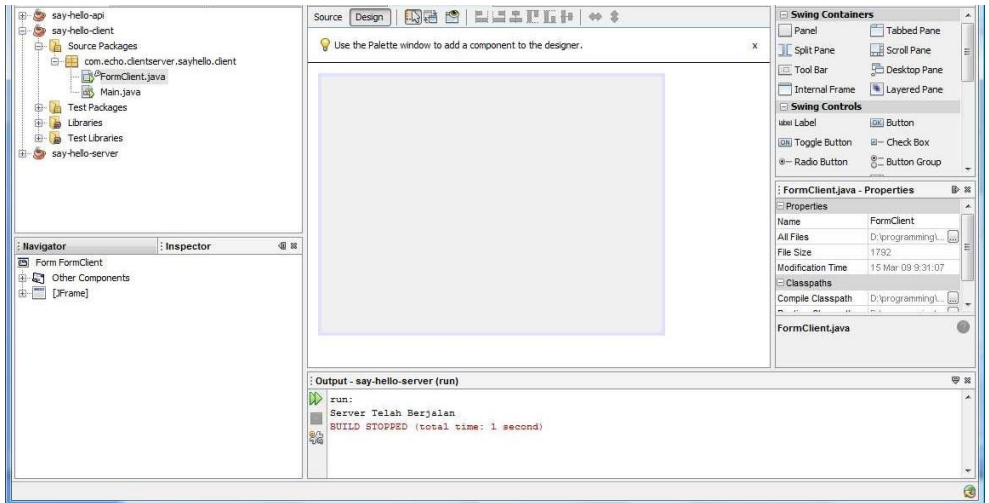
Gambar 117. New File

Lalu klik Next.



Gambar 118. New JFrame From

Pada Class Name beri nama “FormClient” lalu klik Finish.



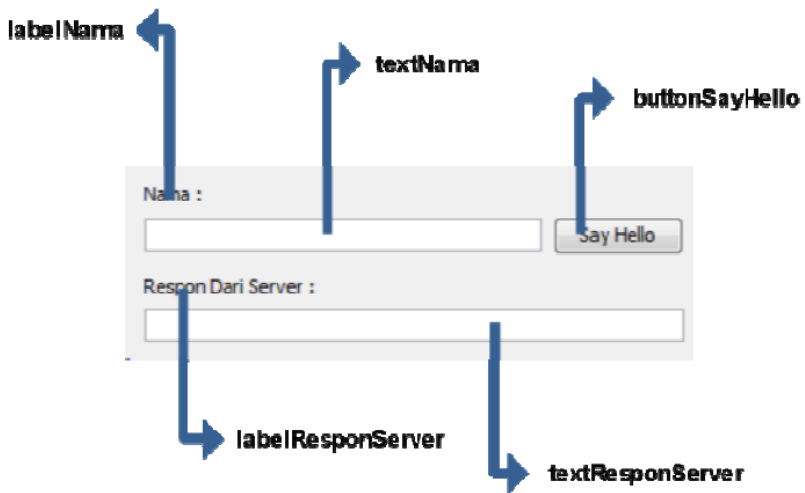
Gambar 119. GUI Pada Netbeans IDE

Buatlah tampilan JFrame tadi menjadi tampilan seperti ini :



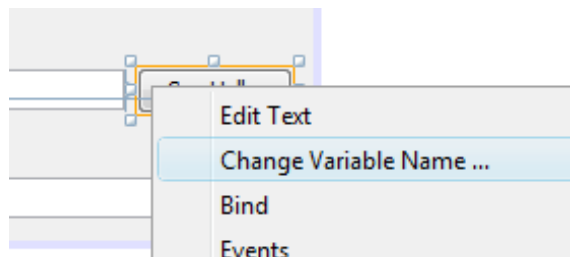
Gambar 120. Tampilan Aplikasi SayHello

Lalu ubah nama Variable nya seperti dibawah ini.



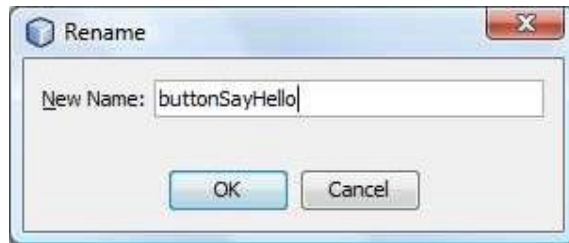
Gambar 121. Nama-Nama Variabel Form Client SayHello

Untuk mengubah nama variable klik kanan, lalu Change Variable Name.



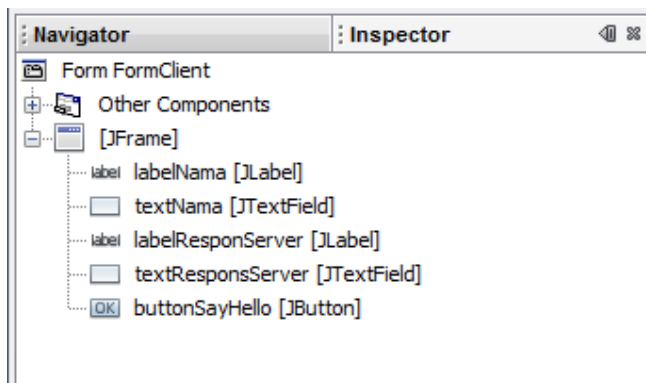
Gambar 122. Mengubah Nama Variabel

Lalu masukkan nama Variable nya di Rename.



Gambar 123. Rename

Kemudian klik OK, maka pada inspector akan berubah seperti ini :



Gambar 124. Inspector

Sekarang tahap terakhir adalah membuat koneksi dari Client ke Server.

SayHello sayHello =

(SayHello)

Naming

lookup("rmi://localhost:1099/sayHello");

Ketikkan kode diatas pada konstruktor FormClient yang sudah kita buat, caranya masuk ke bagian Source. Setelah itu ubah konstruktornya sehingga menjadi seperti ini :

```
private SayHello sayHello;

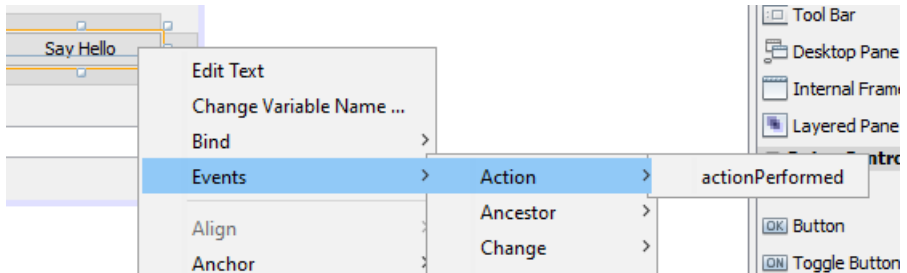
/** Creates new form FormClient */
public FormClient() {
    try {
        sayHello = (SayHello) Naming.lookup("rmi://localhost:1099/sayHello");
    } catch (NotBoundException ex) {
        System.out.println("Terjadi Error Dengan Pesan : " + ex.getMessage());
        System.exit(1);
    } catch (MalformedURLException ex) {
        System.out.println("Terjadi Error Dengan Pesan : " + ex.getMessage());
        System.exit(1);
    } catch (RemoteException ex) {
        System.out.println("Terjadi Error Dengan Pesan : " + ex.getMessage());
        System.exit(1);
    }

    initComponents();
}
```

Gambar 125. Konstruktor Form Client

Setelah itu kembali ke Design, lalu pada ActionListener di buttonSayHello klik kanan

buttonSayHello, lalu pilih Events > Action > actionPerformed.



Gambar 126. Menambah Action Listener

Lalu metode button akan seperti ini:

```
private void buttonSayHelloActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    try {  
        String nama = textNama.getText();  
        String response = sayHello.sayHello(nama);  
        textResponsServer.setText(response);  
    } catch (RemoteException ex) {  
        System.err.println("Terjadi Error Dengan Pesan : " + ex.getMessage());  
        System.exit(1);  
    }  
}
```

Gambar 127. Isi Metode buttonSayHelloActionPerformed

Proses pembuatan Client selesai, sekarang kita ubah class utama (Main) yang tadi dibuat oleh Netbeans untuk project say-hello-client menjadi seperti ini:

```

package com.echo.clientserver.sa hello.client;

import javax.swing.SwingUtilities;

public class Wain {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(new Runnable() {

            public void run() {

                FormClient client = new FormClient();
                client.setVisible(true);

            }

        });

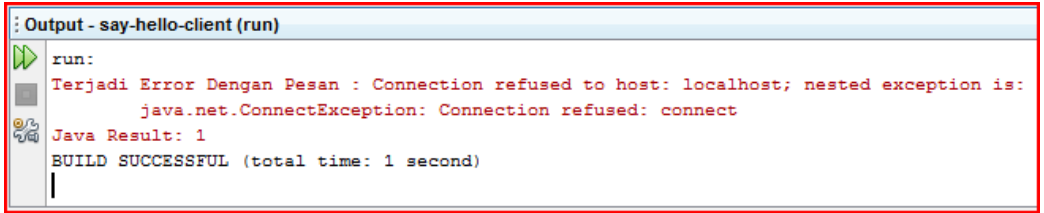
    }

}

```

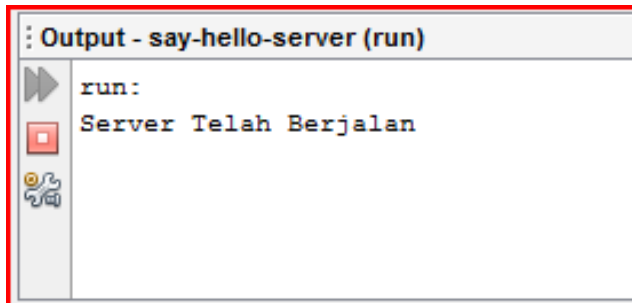
c. Menjalankan Aplikasi SayHello

Setelah membuat Client dan Server untuk aplikasi SayHello, saatnya kita sekarang menjalankan aplikasi SayHello, tapi sebelum dijalankan kita terlebih dahulu jalankan Server-nya, karena jika Client dijalankan terlebih dahulu maka akan terjadi Error seperti gambar dibawah ini :



Gambar 128. Error Client Ketika dijalankan

Jadi kita harus jalankan terlebih dahulu Server-nya, caranya klik kanan pada project say-hello-server lalu klik Run File. Jika Server berhasil berjalan, maka akan terlihat output seperti pada gambar dibawah ini.



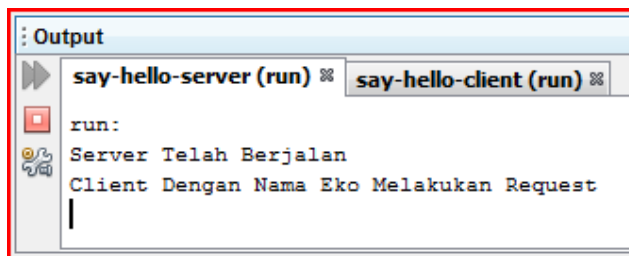
Gambar 129. Output Server

Artinya Server telah berjalan, sekarang tinggal kita jalankan Client, caranya sama seperti menjalankan Server. Sekarang coba tuliskan "Eko" di textNama, lalu klik buttonSayHello, maka server akan merespon dan hasil responnya akan ditampilkan di textResponServer :



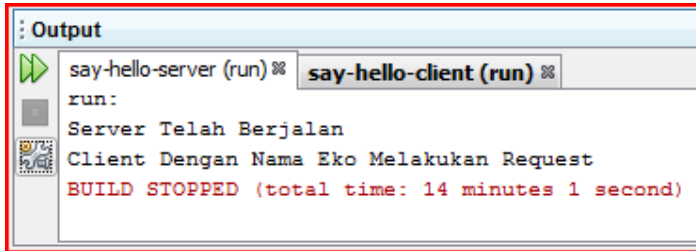
Gambar 130. Repon Dari Server

Dan sekarang kita bisa melihat output responnya di output Server :



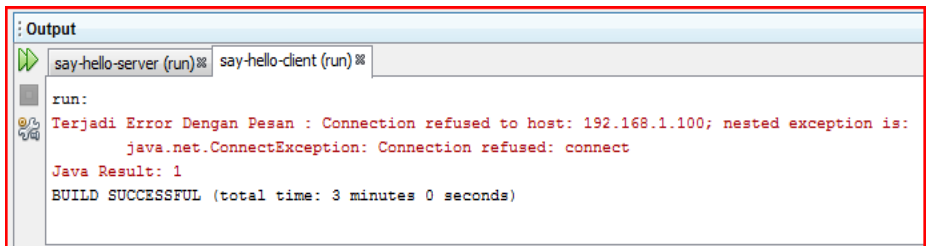
Gambar 131. Trace Respon Server

Ini membuktikan bahwa Client saling berkomunikasi dengan Server, Coba kita matikan server dengan mengklik tombol kotak merah maka akan terlihat seperti ini.



Gambar 132. Mematikan Server

Setelah itu klik kembali buttonSayHello pada aplikasi Client SayHello, maka program akan keluar dan pada trace Client.



Gambar 133. Error Pada Client Ketika Server Mati

Ini berarti memang Client saling berkomunikasi dengan Server, sehingga jika Server mati, maka Client akan ikut mati. Dan kita telah membuat sebuah aplikasi sederhana Client-Server yang saling merespon dari Client dan Server.

C. Latihan

Buatlah contoh Client-Server yang lain!

D. Referensi

Khannedy, K. Eko (2011). Membangun Aplikasi Client-Server Menggunakan Java. Subang: StripBandunk.

Budiharto, W. (2004). Pemrograman Web Menggunakan J2EE. Jakarta: Elexmedia Komputindo.

Rohmah, N. Ratnasari. (2013). Client/Server dengan Java Remote Method Invocation (Java RMI), Sebuah Tutorial, vol. 3, No. 2,